

Assignment 1: Ray Casting

作者：王洵

1 实验原理

1.1 射线投射与最近交点

射线投射 (Ray Casting) 将二维图像上的每个像素对应到三维空间中的一条射线，并求该射线与场景几何体的交点。射线可参数化为

$$\mathbf{p}(t) = \mathbf{o} + t\mathbf{d},$$

其中 $\mathbf{o}$ 为原点， $\mathbf{d}$ 为方向向量， t 为沿射线的标量参数。交点处的 t 同时刻画沿视线方向的深度：在方向已归一化时， $|t|$ 等于原点到交点的欧氏距离。对每个像素在所有交点中取 t 最小者，即可得到可见表面上的最近点，并据此决定该像素的颜色。

1.2 正交投影相机

正交投影相机 (Orthographic Camera) 发出一组互相平行的射线：方向 $\mathbf{d}$ 对所有像素相同，差异仅体现在原点在图像平面上的位置。图像平面由中心 $\mathbf{c}$ 、单位视线方向 $\hat{\mathbf{d}}$ 、单位竖直轴 $\hat{\mathbf{u}}$ 以及边长 s 描述。竖直轴需先去掉沿 $\hat{\mathbf{d}}$ 的分量并归一化，再令水平轴

$$\hat{\mathbf{h}} = \hat{\mathbf{d}} \times \hat{\mathbf{u}},$$

从而构成右手标准正交基 (orthonormal basis)。屏幕坐标 $(u, v) \in [0, 1]^2$ 到射线原点的映射为

$$\mathbf{o}(u, v) = \mathbf{c} + (v - \frac{1}{2})s\hat{\mathbf{u}} + (u - \frac{1}{2})s\hat{\mathbf{h}},$$

对应的射线为 $\mathbf{p}(t) = \mathbf{o}(u, v) + t\hat{\mathbf{d}}$ 。由于平行射线覆盖整个空间，求交时的参数下界 $t_{\min}$ 取充分小的负数，使图像平面前方与后方的表面均可被采样。

1.3 球体代数求交

球体 $|\mathbf{x} - \mathbf{c}_s| = r$ 与射线的代数求交，将 $\mathbf{p}(t)$ 代入球面方程得到关于 t 的二次方程

$$at^2 + bt + c = 0,$$

其中 $a = \mathbf{d} \cdot \mathbf{d}$, $b = 2(\mathbf{o} - \mathbf{c}_s) \cdot \mathbf{d}$, $c = |\mathbf{o} - \mathbf{c}_s|^2 - r^2$ 。判别式 $\Delta = b^2 - 4ac$ 决定是否存在实交点；两根

$$t_{\pm} = \frac{-b \pm \sqrt{\Delta}}{2a}$$

分别对应近端与远端交点。当 $t \geq t_{\min}$ 且该 t 小于当前已记录的最近交点参数时，根据该交点更新最近交点参数及其对应材质。场景含多个图元时，对同一射线依次与各图元求交，始终保留最小的 t ，从而沿视线区分前后遮挡。

1.4 常数着色与深度可视化

着色采用常数漫反射颜色（diffuse color）：命中物体后，将该物体材质的漫反射颜色直接赋给像素。深度可视化则将最近交点的参数 t 线性映射到灰度。给定区间 $[t_{\text{near}}, t_{\text{far}}]$ ，令

$$g = \text{clamp}\left(\frac{t_{\text{far}} - t}{t_{\text{far}} - t_{\text{near}}}, 0, 1\right),$$

则较近处偏白、较远处偏黑；区间外的深度截断到端点，从而把三维深度结构编码为二维灰度图。

2 程序设计与实现

2.1 总体架构

系统由线性代数库、图像读写库、场景解析模块、几何与相机模块以及渲染主程序组成：

- 场景解析模块读入场景描述，构造相机、背景色、材质与物体层次；
- 几何模块以求交接口组织球体与物体组；
- 相机模块生成正交投影射线；
- 主程序解析命令行，逐像素求交并写出颜色图与深度图。

读入场景后，主循环对每个像素生成射线并与顶层物体组求交：

```
Ray ray = camera->generateRay(Vec2f(u, v));
Hit hit(max_t, NULL);
bool intersected = group->intersect(ray, hit, camera->getTMin());
```

其中初始交点参数取很大正数，使首次命中即可写入最近交点记录。

2.2 图元抽象与球体求交

三维图元接口保存材质指针，并声明与射线的求交方法：

```

class Object3D {
public:
    Object3D() : material(NULL) {}
    virtual ~Object3D() {}
    virtual bool intersect(const Ray &r, Hit &h, float tmin) = 0;
protected:
    Material *material;
};

```

球体在此基础上存储球心与半径。求交时构造二次方程系数并计算判别式；若存在实根，则对近端与远端分别检验，并在更近时写入交点信息：

```

float a = dir.Dot3(dir);
float b = 2.0f * oc.Dot3(dir);
float c = oc.Dot3(oc) - radius * radius;
float discriminant = b * b - 4.0f * a * c;
if (discriminant < 0.0f)
    return false;
float t = (-b - sqrt_disc) / (2.0f * a);
if (t >= tmin && t < h.getT()) {
    h.set(t, material, r);
    hit = true;
}

```

写入交点时同时记录参数 t 、材质，以及由 $\mathbf{o} + t\mathbf{d}$ 得到的交点空间坐标。

2.3 物体组

物体组用指针数组保存子物体，构造时按物体个数分配，再按索引填入。求交时遍历全部子物体；由于各子物体在更新交点记录时只保留更小的 t ，遍历结束后得到整组最近交点：

```

bool Group::intersect(const Ray &r, Hit &h, float tmin) {
    bool hit = false;
    for (int i = 0; i < numObjects; i++) {
        if (objects[i] != NULL && objects[i]->intersect(r, h, tmin))
            hit = true;
    }
    return hit;
}

```

2.4 正交相机

构造时归一化视线方向，将上方向减去其在视线上的分量后再归一化，并用叉积得到图像平面水平轴：

```

direction /= direction.Length();
up -= direction * up.Dot3(direction);
up /= up.Length();
Vec3f::Cross3(horizontal, direction, up);
horizontal /= horizontal.Length();

```

生成射线时按屏幕坐标在图像平面上插值原点，方向对所有像素保持不变：

```

Vec3f origin = center
    + (v - 0.5f) * size * up
    + (u - 0.5f) * size * horizontal;
return Ray(origin, direction);

```

最小交点参数返回约 -10^{30} ，使正交相机从无穷远一侧进入场景。

2.5 主渲染循环

主程序解析输入场景、图像尺寸、颜色输出以及深度映射近远界与深度图路径。对每个像素，将像素中心映射到 $[0, 1]^2$ ；宽高比不为 1 时对水平或竖直坐标做裁剪，使视场不被拉伸。命中后写入漫反射颜色，并将 t 映射为灰度：

```

image.SetPixel(x, y, hit.getMaterial()->getDiffuseColor());
float gray = (depth_max - hit.getT()) / (depth_max - depth_min);
if (gray < 0.0f) gray = 0.0f;
if (gray > 1.0f) gray = 1.0f;
depthImage.SetPixel(x, y, Vec3f(gray, gray, gray));

```

最后分别保存颜色图与深度图。

3 实验结果

3.1 测试命令

七组场景均在 200×200 分辨率下渲染。典型命令形如：

```

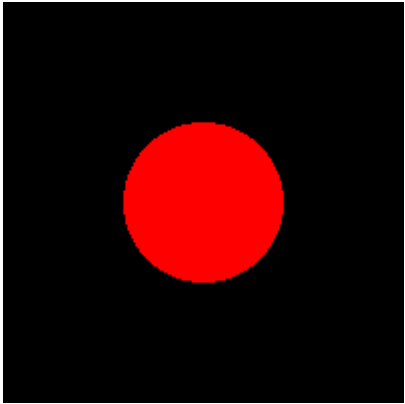
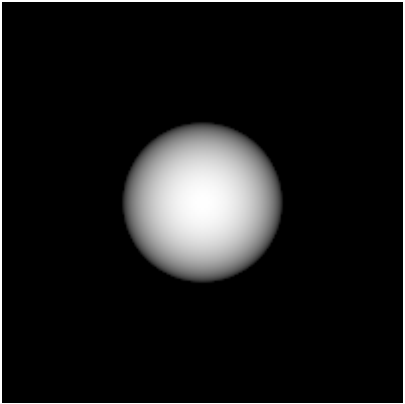
# -input 输入场景; -size 图像宽高; -output 颜色输出
# -depth 深度映射近远界与深度图路径
raytracer -input <单球场景> -size 200 200 -output output1_01.tga -depth 9 10 depth1_01.tga

```

3.2 单球场景

参数：红色单位球位于原点，正交相机沿 $-z$ 观察，深度映射区间 $[9, 10]$ 。

图 3.2-1 ~ 3.2-2 单球颜色图与深度图

图 3.2-1 颜色 (漫反射红)	图 3.2-2 深度 ([9, 10])
	

颜色图呈现圆形色块，未命中区域为背景。深度图在球面上由中心到边缘呈径向灰度变化：球心附近 t 较小、偏白，边缘 t 较大、偏暗，说明正交平行射线下球面深度分布符合几何预期。

3.3 多球与遮挡

随球体数目、位置与材质颜色变化，颜色图显示最近表面的漫反射色；深度图在重叠区域保留更小 t 对应的灰度。

图 3.3-1 ~ 3.3-2 五球场景 (深度区间 [8, 12])

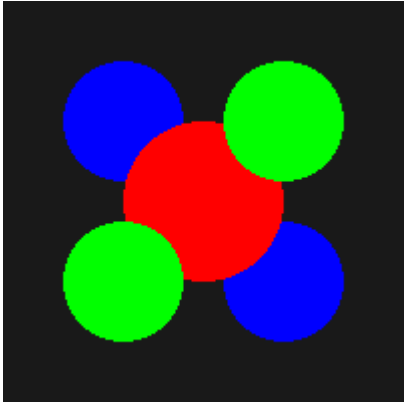
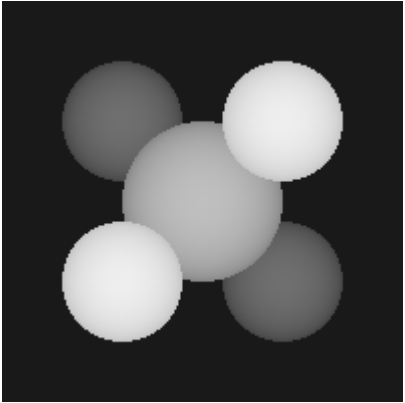
图 3.3-1 颜色	图 3.3-2 深度
	

图 3.3-3 ~ 3.3-4 多球场景 (深度区间 [8, 12])

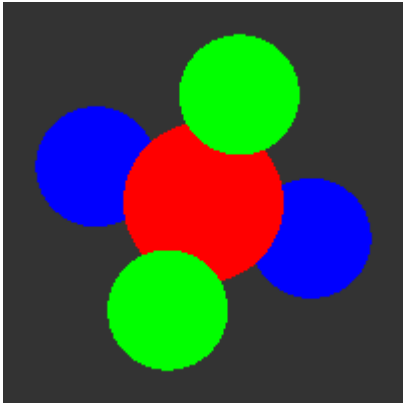
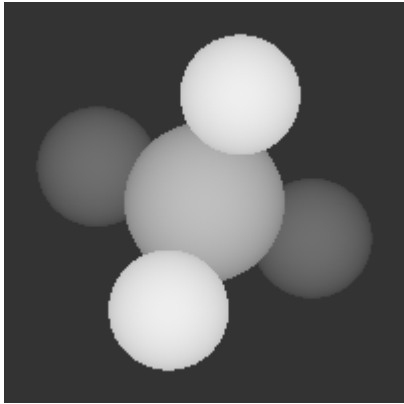
图 3.3-3 颜色	图 3.3-4 深度
	

图 3.3-5 ~ 3.3-6 多球场景 (深度区间 $[12, 17]$)

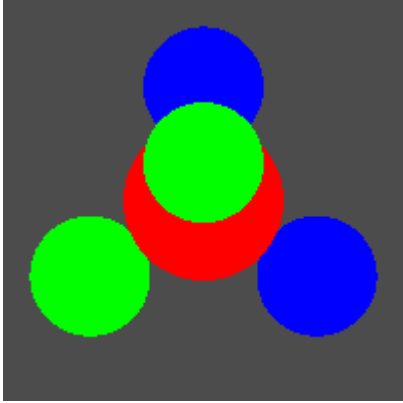
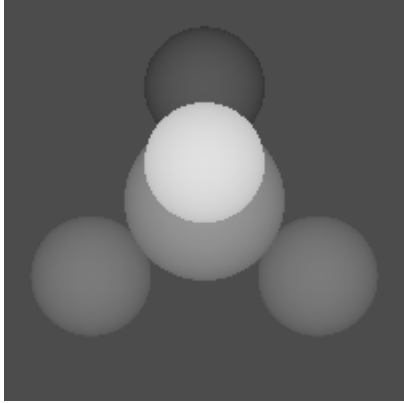
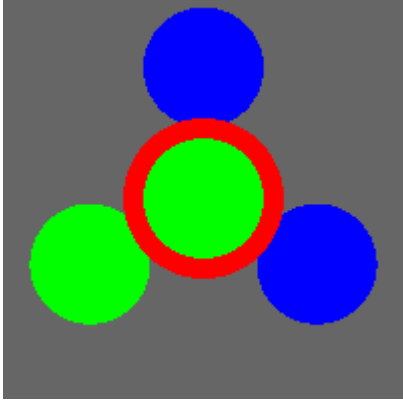
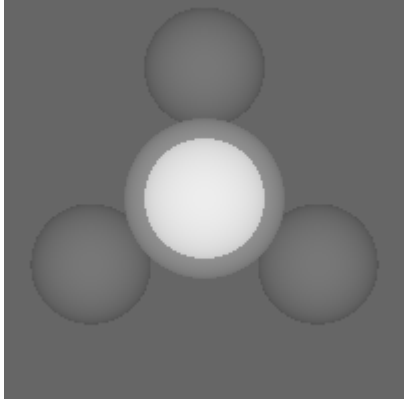
图 3.3-5 颜色	图 3.3-6 深度
	

图 3.3-7 ~ 3.3-8 多球场景 (深度区间 $[14.5, 19.5]$)

图 3.3-7 颜色	图 3.3-8 深度
	

重叠处颜色取自前方物体，深度灰度亦对应更近表面，说明物体组遍历求交后按最小 t 选择可见表面。

3.4 近距与跨图像平面配置

图 3.4-1 ~ 3.4-2 近距三球（深度区间 $[3, 7]$ ）

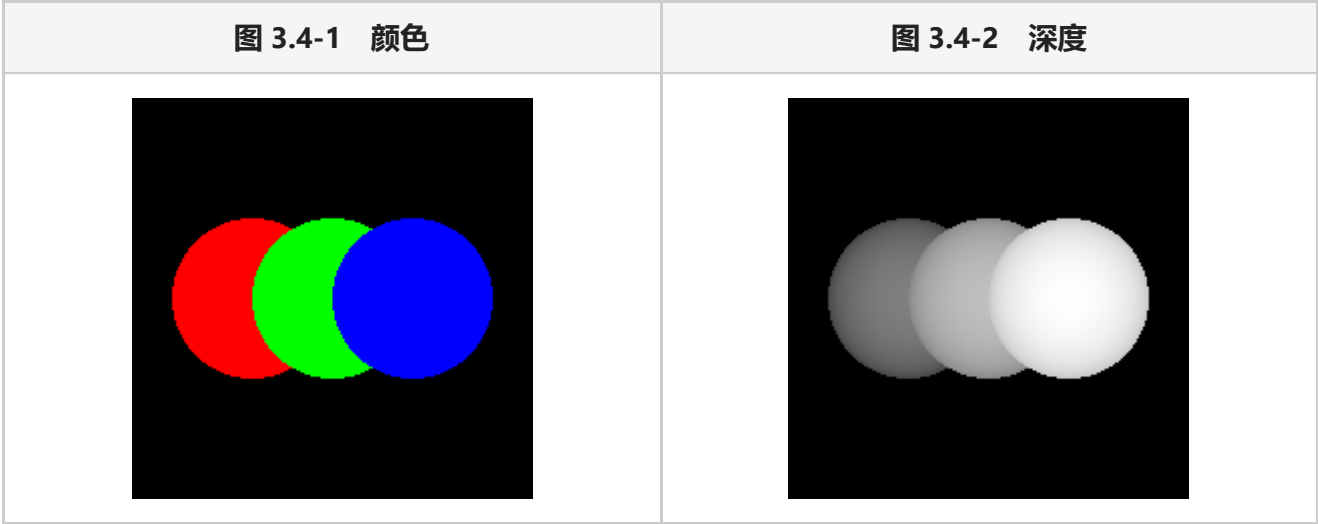
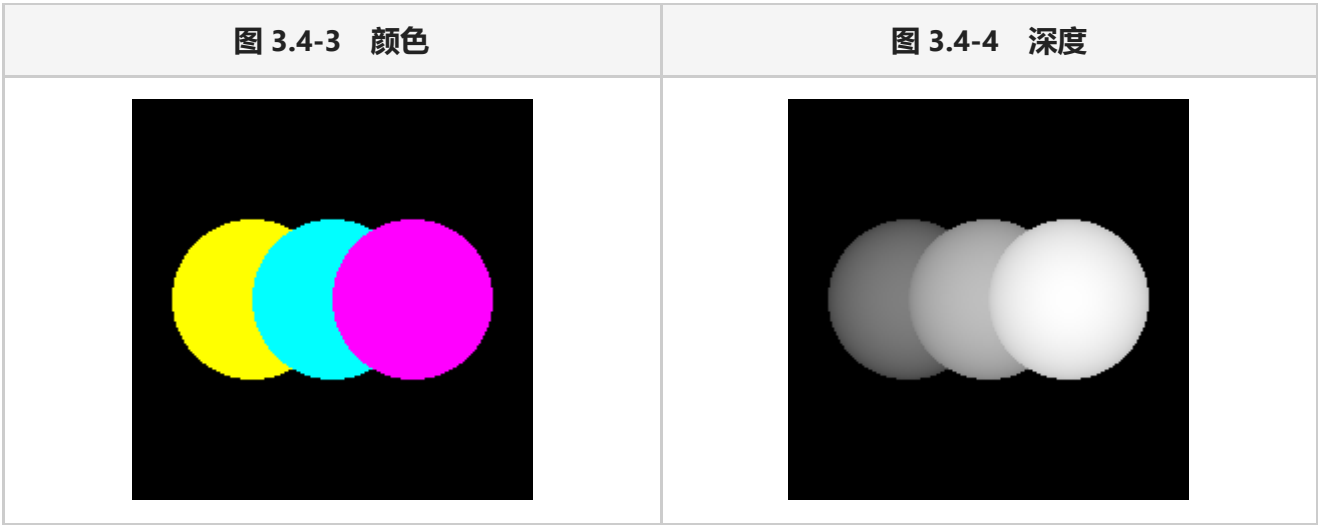


图 3.4-3 ~ 3.4-4 跨平面三球（深度区间 $[-2, 2]$ ）



近距场景中球体占据画面更大比例，深度对比集中在较小 t 区间。跨平面场景中相机位于原点附近，球体分布在图像平面前后两侧；因 $t_{\min}$ 取大负数，前后两侧表面均可被采样，深度映射区间 $[-2, 2]$ 下灰度仍能区分远近，和正交求交的预期相符。

4 问题记录

首轮深度图与样例的明暗方向相反：球心等近处偏黑、边缘等远处偏白。原因在于灰度映射写成了 $(t - t_{\text{near}})/(t_{\text{far}} - t_{\text{near}})$ ，而样例约定近处偏白、远处偏黑。将映射翻转后问题解决。

解析器没有完全识别场景文件中的材质关键字，读入时直接报未知标记并退出。在材质解析分支中完成关键字拓展后，问题解决。另，解析器中材质数组分配写成了旧式 `new (Material*)[n]`，在较新的 g++ 下无法通过编译，改为 `new Material*[n]` 后解决。

主程序编译时会因 Camera、Group、Material 仅为前置声明而出现不完整类型错误。补入相机、物体组与材质头文件后问题解决。

5 总结

本实验实现了基于正交投影的射线投射器：由屏幕坐标在图像平面上生成平行射线，对球体用二次方程求交，经物体组遍历取最小 t 作为可见表面，再以常数漫反射色着色，并将交点参数线性映射为深度灰度图。图像平面标准正交基的构造、交点参数的比较与更新，以及深度近远区间的设定，共同决定了颜色与深度两类输出的形态。

附录 (TODO list 记录)

- ☒ 实现三维图元抽象接口，保存材质并声明求交方法
- ☒ 实现球体图元，用代数二次方程求交，并在更近时更新交点参数与材质
- ☒ 实现物体组，按索引加入子物体并遍历求交以得到最近交点
- ☒ 实现正交相机，由图像中心、视线、上方向与边长构造标准正交基并生成平行射线
- ☒ 使用场景解析模块加载相机、背景色与场景物体
- ☒ 在主程序中解析命令行，逐像素生成射线、求交并写出颜色图
- ☒ 实现深度可视化，将交点参数线性映射为灰度并保存深度图
- ☐ Extra credit: 同时实现几何与代数球体求交、添加圆柱与圆锥、基于到图像平面距离的雾效等